Exercise on Cost-effective Regression Testing
Week 5: Regression Testing

Below are example solutions to the exercises. While these address the requirements, you are encouraged to explore alternative approaches and solutions.
Your creativity and problem-solving strategies are valued :)
(Solutions for the optional exercises are not provided)

Example Case: MyMovieApp Application
Here we will use as a case a hypothetical set of test cases for a web application to register and retrieve data about movies and their cast members (actors and actresses). The app supports the addition of movie and cast entries manually or automatically by providing a URL to the corresponding item on IMDB (International Movie Database).

For the sake of simplicity we chose **10 test cases** exercising distinct, yet similar scenarios of the application. Each test case has a title (ID), a set of preconditions and postconditions, and finally a sequence of steps. The test cases also use data that can be stored in a file and automatically replaced when building a test plan. The data used in test cases are marked between angled brackets "<< >>".

You are not expected to change, create or analyse details of the test cases. Rather, you will use them, as they are, to execute the techniques. Here we want to select, minimise and classify the set of test cases to reduce costs in testing. **The test artefacts can be found in the appendix to this exercise.**

<u>Part 1 - Test Case Selection and Prioritization</u>

We begin with the Diversity-based test selection (DBT). Use the JaccardIndex to analyse the text diversity of steps, preconditions and postconditions of the tests all together. For simplicity, implement a script that parses the text information and calculates the JaccardIndex as words (alternatively, you can find and use string libraries that calculate Jaccard index using the actual algorithm based on q-grams)

Q1.1 - **Calculate all similarity values** and fill in the similarity matrix below. We suggest a precision of two digits of precision for the similarity values (e.g.: 0.78).

Remember: You can ignore the diagonal and focus only on the upper diagonal of the matrix, since the similarity function is symmetrical.

Similarity Matrix:

	TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8	TC9	TC10
TC1	-	0.52	0.24	0.22	0.22	0.22	0.24	0.2	0.26	0.38
TC2	-	-	0.21	0.19	0.19	0.2	0.18	0.16	0.19	0.23
TC3	-	-	-	0.79	0.79	0.69	0.7	0.56	0.59	0.33
TC4	-	-	-	-	0.64	0.72	0.63	0.54	0.53	0.29
TC5	-	-	-	-	-	0.76	0.56	0.62	0.5	0.29
TC6	-	-	-	-	-	-	0.54	0.57	0.51	0.3
TC7	-	-	-	-	-	-	-	0.74	0.73	0.32
TC8	-	-	-	-	-	-	-	-	0.61	0.27
TC9	-	-	-	-	-	-	-	-	-	0.35
TC10	-	-	-	-	-	-	-	-	-	-

Q1.2 – Based on the matrix, prioritise the test suite by executing first the tests that are more diverse to the others. Then provide **3 subsets of test cases,** covering the top 20%, 40%, 60% of the most diverse tests. For instance, you can write only the ID of the selected test cases Ex: T_20 = {TC1, TC8}. Use two methods for identifying the subsets: i) Similarity-based test case prioritization and ii) Distance-based test case prioritization (Max-min method).

Option1: Similarity-based test case prioritization.

Step 1: Start with the Full Test Suite

Initial test suite:

```
T’ = {TC1, TC2, TC3, TC4, TC5, TC6, TC7, TC8, TC9, TC10}
```

Step 2: Select the Top 60% of Test Cases

To select the top 60% (i.e., 6 test cases), we need to remove the 4 most similar test cases from the initial set.

Identify the most similar test cases:

- TC3 and TC4 have a similarity of 0.79; TC3 and TC5 also have a similarity of 0.79 => Randomly remove TC3.

```
T’ = {TC1, TC2, TC4, TC5, TC6, TC7, TC8, TC9, TC10}
```

Find the next most similar test cases:

- TC5 and TC6 have a similarity of 0.76. => Randomly remove TC6.

`T' ={TC1, TC2, TC4, TC5, TC7, TC8, TC9, TC10}`

Continue removing based on similarity:

- TC7 and TC8 have a similarity of 0.74. => Randomly remove TC7.

`T' = {TC1, TC2, TC4, TC5, TC8, TC9, TC10}`

Next highest similarity:

- TC5 and TC4 have a similarity of 0.64 => Randomly remove TC5.

`T' = {TC1, TC2, TC4, TC8, TC9, TC10}`

Top 60% subset:

`T_60 = {TC1, TC2, TC4, TC8, TC9, TC10}`

Step 3: Select the Top 40% of Test Cases

Repeat the same process to select the top 40% (i.e., 4 test cases).
Starting with `T' = {TC1, TC2, TC4, TC8, TC9, TC10}`:
Remove 2 cases based on highest similarity.

- Remove TC9 and TC8 based on similarity.

Top 40% subset:

`T_40 ={TC1, TC2, TC4, TC10}`

Step 4: Select the Top 20% of Test Cases

Repeat the same process to select the top 40% (i.e., 4 test cases).
Starting with `T' = {TC1, TC2, TC4, TC10}`:
Remove 2 cases based on highest similarity.

- Remove TC2 and TC10 based on similarity.

Top 20% subset:

`T_20 = {TC1, TC4}`

Selected subsets:

T_60 = {TC1, TC2, TC4, TC8, TC9, TC10} .
T_40 = {TC1, TC2, TC4, TC10}
T_20 = {TC1, TC4}

Option2: Distance-based test case prioritization (Max-min method).

Distance Matrix

To calculate the distance values, subtract the similarity value from 1, as they are complementary.

	TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8	TC9	TC10
TC1	-	0.48	0.76	0.78	0.78	0.78	0.76	0.8	0.74	0.62
TC2	-	-	0.79	0.81	0.81	0.8	0.82	0.84	0.81	0.77
TC3	-	-	-	0.21	0.21	0.31	0.3	0.44	0.41	0.67
TC4	-	-	-	-	0.36	0.28	0.37	0.46	0.47	0.71
TC5	-	-	-	-	-	0.24	0.44	0.38	0.5	0.71
TC6	-	-	-	-	-	-	0.46	0.43	0.49	0.7
TC7	-	-	-	-	-	-	-	0.26	0.27	0.68
TC8	-	-	-	-	-	-	-	-	0.39	0.73
TC9	-	-	-	-	-	-	-	-	-	0.65
TC10	-	-	-	-	-	-	-	-	-	-

Step0: We start with an empty prioritized test suite `P={}`

Step1: Find the highest distance value

(TC8, TC2) have the highest distance value 0.84. We add both test cases to the test suite `P={TC8, TC2}`

Then we iterate over the rest of tests (individually) picking one at a time.

Step2: Find the **minimal** distance between each of the remaining test cases and the picked ones (TC8, TC2).

- TC1: TC8 distance = 0.8; TC2 distance = 0.48. Minimal is 0.48
- TC2: -
- TC3: TC8 distance = 0.44; TC2 distance = 0.79. Minimal is 0.44
- TC4: TC8 distance = 0.46; TC2 distance = 0.81. Minimal is 0.46
- TC5: TC8 distance = 0.38; TC2 distance = 0.81. Minimal is 0.38
- TC6: TC8 distance = 0.43; TC2 distance = 0.8. Minimal is 0.43
- TC7: TC8 distance = 0.26; TC2 distance = 0.82. Minimal is 0.26
- TC8: -
- TC9: TC8 distance = 0.39; TC2 distance = 0.81. Minimal is 0.39

- TC10: TC8 distance = 0.73; TC2 distance = 0.77. Minimum is 0.73

Step3: Pick the test case with the **maximal** distance from step 2. In this case, it is TC10 with a distance of 0.73.

Step4: Add the picked test case to the prioritized test suite: `P={TC8, TC2, TC10}`

Repeat the Steps 2-4 (now you need to compare distances to TC8, TC2, TC10). As a result, TC1 should be selected. `P={TC8, TC2, TC10, TC1}`

Repeat the Steps 2-4 (now you need to compare distances to TC8, TC2, TC10, TC1). As a result, TC4 should be selected. `P={TC8, TC2, TC10, TC1, TC4}`

Keep repeating Steps 2-4 until there are no test cases left.

The final prioritized test suite:
`P={TC8, TC2, TC10, TC1, TC4, TC9, TC5, TC7, TC6, TC3}`

Selected subsets:

T_60 = {TC8, TC2, TC10, TC1, TC4, TC9} .
T_40 = {TC8, TC2, TC10, TC1}
T_20 = {TC8, TC2}

Q1.3 - How do we compare which prioritised test suite is better? After executing and debugging the 10 test cases, we find the traceability below between test cases and faults (not failures!). Reflect on how many faults are revealed by the different subsets (T_60, T_40, T_20). You can choose which subsets to use T_60, T_40, T_20 from Q1.2 similarity based method or T_60, T_40, T_20 from Q1.2 distance based method.

	F1	F2	F3	F4	F5	F6
TC1	x		x			
TC2						
TC3		x				x
TC4						
TC5	x	x				x
TC6						
TC7			x			
TC8					x	

TC9				x		x
TC10	x					x

From the Q1.2 (similarity method) we have the following subsets:
T_60 = {TC1, TC2, TC4, TC8, TC9, TC10} .
T_40 = {TC1, TC2, TC4, TC10}
T_20 = {TC1, TC4}

T_60
Executing TC1 reveals 2 faults: F1 and F3
Executing TC2 reveals no additional faults
Executing TC4 reveals no additional faults
Executing TC8 reveals 1 additional fault: F5
Executing TC9 reveals 2 additional faults: F4 and F6
Executing TC10 reveals no additional faults (faults F1 and F6 have already been revealed by TC1 and TC9)

Total faults revealed by T_60: 5 faults (F1, F3, F4, F5, F6).

TC_40
Executing TC1 reveals 2 faults: F1 and F3
Executing TC2 reveals no additional faults
Executing TC4 reveals no additional faults
Executing TC10 reveals 1 additional fault: F6 (fault F1 has already been revealed by TC1)

Total faults revealed by T_40: 3 faults (F1, F3, F6).

TC_20
Executing TC1 reveals 2 faults: F1 and F3
Executing TC4 reveals no additional faults

Total faults revealed by T_20: 2 faults (F1, F3).

From this, we observe that T_60 is the most effective as it reveals the largest number of faults (5), followed by T_40 with 3 faults, and finally T_20 with 2 faults. This demonstrates that a larger subset of test cases generally has a higher chance of revealing more faults. However, this outcome can vary depending on the specific case.

Depending on the available time and resources, smaller subsets like T_40 or T_20 can still be useful for prioritising critical faults (F1 and F3).

Q1.4 - A colleague suggested a different prioritised test suite: P-Sam: {TC8, TC6, TC2, TC3, TC4, TC7, TC1, TC5, TC2, TC9}. Which prioritised test suite is better at revealing faults earlier (APFD)? Justify why a test suite is better than the other. Compare three prioritized test suites: Test suite prioritised by similarity method, Test suite prioritised by distance (max-min method) and P-Sam test suite.

Test suite prioritised by similarity method:
{TC1, TC2, TC4, TC8, TC9, TC10, TC5, TC7, TC6, TC3}

Test suite prioritised by distance (max-min method):
{TC8, TC2, TC10, TC1, TC4, TC9, TC5, TC7, TC6, TC3}

P-Sam test suite:
{TC8, TC6, TC2, TC3, TC4, TC7, TC1, TC5, TC10, TC9}.

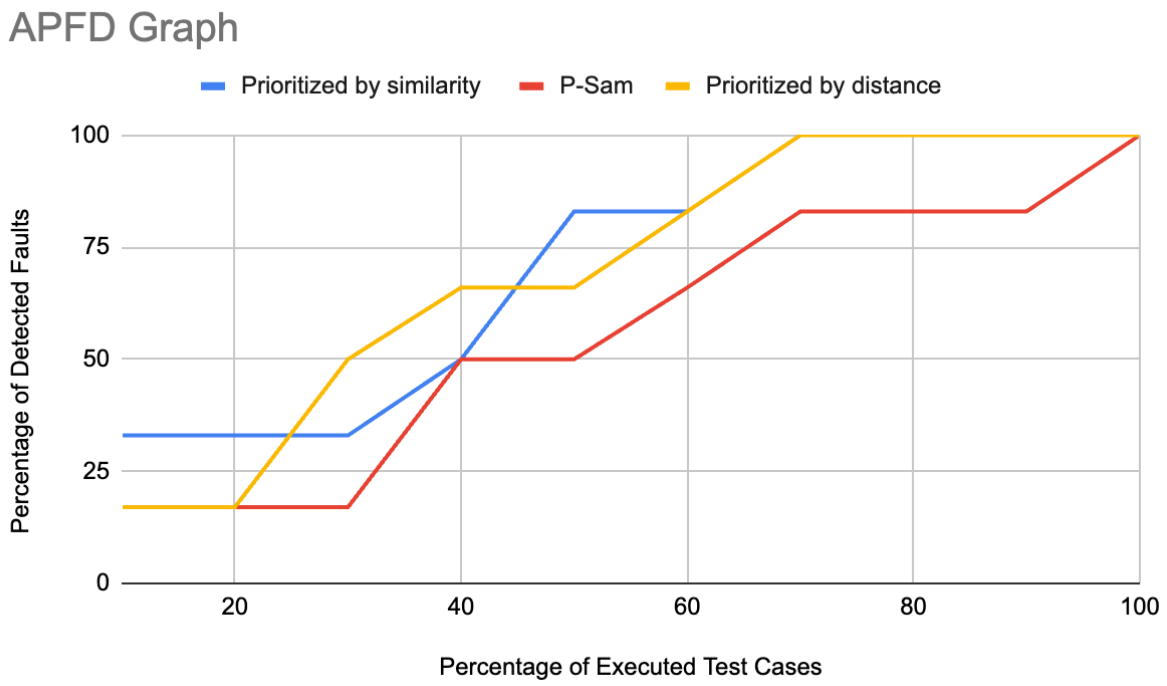
Comments on the Table:

- There are 6 faults in total, so detecting 1 fault corresponds to approximately 17% of the faults (100% / 6 ≈ 17%).
- Each test suite contains 10 tests, so executing 1 test corresponds to 10% of the total tests (100% / 10 = 10%).

Test suite	**Percentage of Detected Faults**									
Prioritized by similarity	33 (TC1)	33 (TC2)	33 (TC4)	50 (TC8)	83 (TC9)	83 (TC10)	100 (TC5)	100 (TC7)	100 (TC6)	100 (TC3)
P-Sam	17 (TC8)	17 (TC6)	17 (TC2)	50 (TC3)	50 (TC4)	66 (TC 7)	83 (TC1)	83 (TC5)	83 (TC10)	100 (TC9)
Prioritized by distance	17 (TC8)	17 (TC2)	50 (TC10)	66 (TC1)	66 (TC4)	83 (TC9)	100 (TC5)	100 (TC7)	100 (TC6)	100 (TC3)

Average Percentage of Faults Detected (APFD) is a metric used to measure the rate of fault detection during test prioritization. To visually compare the APFD of different test suites, we can plot line graphs where:

- The x-axis represents the percentage of executed test cases.
- The y-axis represents the percentage of detected faults.



The APFD value can be approximated by the area under the curve (AUC). The larger the area, the more effective the test suite is at early fault detection. There is also a specific formula for its calculation (see the link if you are interested). A higher APFD indicates that a test suite detects faults earlier in the execution process, which is desirable for efficient testing.

APFD Results:
Prioritized by Similarity: 0.665 (66.5%)
P-Sam: 0.516 (51.6%)
Prioritized by Distance: 0.649 (64.9%)

The test suites prioritized by similarity and distance have very similar APFD values (66.5% and 64.9%, respectively), indicating that both are effective at early fault detection. In contrast, the P-Sam suite has a significantly lower APFD value (51.6%), suggesting that it is less effective at detecting faults early.

Q1.5 – Explain how different selection percentages (T_20, T_40, T_60) for the prioritized test suite impact the coverage of the scenarios represented by the 10 test cases. For example, assume each test case takes approximately 5 minutes to execute. How much time is required to run each test suite? How many faults does each suite detect? Discuss the trade-off between test suite execution time and the number of faults detected.

From the Q1.2 (similarity method) we have the following subsets:
T_60 = {TC1, TC2, TC4, TC8, TC9, TC10} .

T_40 = {TC1, TC2, TC4, TC10}
T_20 = {TC1, TC4}

T_20 (10 minutes, 2 faults):

- This subset has the shortest execution time but detects only 2 faults.
- It is suitable when time and resources are extremely limited, but it provides lower fault coverage.

T_40 (20 minutes, 3 faults):

- This subset provides a balanced trade-off between execution time and fault detection.
- It detects 1 additional fault compared to T_20, at the cost of 10 more minutes of execution time.

T_60 (30 minutes, 5 faults):

- This subset requires the most time but detects the highest number of faults (5 out of 6).
- It is the most effective in terms of fault coverage but requires a longer execution time, which may not always be feasible.

Appendix: Exercise “Option 2 – Manual Test Optimization”

Jaccard Index: This is a similarity score for how much “overlap” in tokens two strings have. To obtain the distance, you need to do: `JaccDist(TCi, TCj) = 1 - J(TCi,TCj)`

$$J(TC_i, TC_j) = \frac{|TC_i \cap TC_j|}{|TC_i \cup TC_j|}$$

1. Manual Test Cases

TC1: Valid Login
Pre-condition:
User <<valid_user>> is registered App login screen is shown
Steps
Access the MyMovieApp
In the login field, type <<valid_user.email>>
In the password field, type << valid_user.pwd>>
Press the "Login" button
Post-condition:
App home screen is shown

TC2: Invalid Login
Pre-condition:
User <<invalid_user>> is not registered
Steps
Access the MyMovieApp
In the login field, type <<invalid_user.mail>>
In the Password field, type <<invalid_user.pwd>>
Press the "Login" button
Post-condition:
Message “User not registered” is shown

TC3: Create movie entry (manual)
Pre-condition:

<<user>> is logged in successfully <<movie>> was not created yet in <<user>>’s profile App home screen is shown
Steps
Click the "Add movie" button
Choose the "manual entry" feature
In the title field type, <<movie>>
Press the " Add" button
Post-condition:
Message "Movie added successfully" is shown Updated table of movies is shown in main screen

TC4: Create movie entry (IMDB)
Pre-condition:
<<user>> is logged in successfully <<movie>> was not created yet in <<user>>’s profile App home screen is shown <<movie_url>> is accessible via Internet
Steps
Click the "Add movie" button
Choose the "Import from IMDB" feature
In the title field type, <<movie_url>>
Press the " Add" button
Post-condition:
Message "Movie added successfully" is shown Updated table of movies is shown in main screen

TC5: Create cast member entry (manual)
Pre-condition:
<<user>> is logged in successfully <<cast>> was not created yet in <<user>>’s profile App home screen is shown
Steps
Click the "Add movie actor/actress" button
Choose the "manual entry" feature
In the Name field type, <<cast.name>>
In the Movie Title field type, <<cast.movies>>
Press the "Add" button
Post-condition:

Updated table of cast members is shown in main screen Message "Actor/actress added successfully" is shown

TC6: Create cast member entry (IMDB)
Pre-condition:
<<user>> is logged in successfully <<cast>> was not created yet in <<user>>'s profile App home screen is shown
Steps
Click the "Add movie actor/actress" button
Choose the "Import from IMDB" feature
In the URL field type, <<cast.url>>
Press the " Add" button
Post-condition:
Updated table of cast members is shown in main screen Message "Actor/actress added successfully" is shown

TC7: Delete a movie entry (by title)
Pre-condition:
<<user>> is logged in successfully <<movie>> was created in <<user>>'s profile App home screen is shown
Steps
Click the "Delete a movie" button
Choose the "By title" feature
In the title field type, <<movie.title>>
Press the " Delete" button
Post-condition:
Updated table of movies is shown in main screen Dialog shows the "Movie deleted successfully" message

TC8: Delete a movie entry (by cast)
Pre-condition:
<<user>> is logged in successfully At least one title from << cast.movies>> was created in <<user>>'s profile <<cast>> was created in <<user>>'s profile App home screen is shown
Steps

Click the "Delete a movie" button
Choose the "By cast member" feature
In the actor/actress field type, <<cast.name>>
Press the "Delete" button
Post-condition:
Updated table of movies is shown in main screen Dialog shows the "Movie deleted successfully" message

TC9: Search movie entries (by year)
Pre-condition:
<<user>> is logged in successfully <<movies>> was created in <<user>>'s profile App home screen is shown
Steps
Click the "Search a movie" button
In the Year field type, <<movie.year>>
Press the "Search" button
Post-condition:
Updated table of movies is shown in main screen Dialog shows the "Movie deleted successfully" message

TC10: Logout
Pre-condition:
<<user>> is logged in successfully App home screen is shown
Steps
Click the "Logout" button
Press the "Confirm" button
Post-condition:
App login screen is shown

2. Set of words in each test case.

TC1	[<<valid_user.email>>, <<valid_user.pwd>>, <<valid_user>>, access, app, button, field, home, in, is, login, mymovieapp, password, press, registered, screen, shown, the, type, user]

TC2	[<<invalid_user.mail>>, <<invalid_user.pwd>>, <<invalid_user>>, access, button, field, in, is, login, message, mymovieapp, not, password, press, registered, registered”, shown, the, type, user, “user]
TC3	[<<movie>>, <<user>>, <<user>>’s, add, added, app, button, choose, click, created, entry, feature, field, home, in, is, logged, main, manual, message, movie, movies, not, of, press, profile, screen, shown, successfully, table, the, title, type, updated, was, yet]
TC4	[<<movie>>, <<movie_url>>, <<user>>, <<user>>’s, accessible, add, added, app, button, choose, click, created, feature, field, from, home, imdb, import, in, internet, is, logged, main, message, movie, movies, not, of, press, profile, screen, shown, successfully, table, the, title, type, updated, via, was, yet]
TC5	[<<cast.movies>>, <<cast.name>>, <<cast>>, <<user>>, <<user>>’s, actor/actress, add, added, app, button, cast, choose, click, created, entry, feature, field, home, in, is, logged, main, manual, members, message, movie, name, not, of, press, profile, screen, shown, successfully, table, the, title, type, updated, was, yet]
TC6	[<<cast.url>>, <<cast>>, <<user>>, <<user>>’s, actor/actress, add, added, app, button, cast, choose, click, created, feature, field, from, home, imdb, import, in, is, logged, main, members, message, movie, not, of, press, profile, screen, shown, successfully, table, the, type, updated, url, was, yet]
TC7	[<<movie.title>>, <<movie>>, <<user>>, <<user>>’s, a, app, button, by, choose, click, created, delete, deleted, dialog, feature, field, home, in, is, logged, main, message, movie, movies, of, press, profile, screen, shown, shows, successfully, table, the, title, type, updated, was]
TC8	[<<cast.movies>>, <<cast.name>>, <<cast>>, <<user>>, <<user>>’s, a, actor/actress, app, at, button, by, cast, choose, click, created, delete, deleted, dialog, feature, field, from, home, in, is, least, logged, main, member, message, movie, movies, of, one, press, profile, screen, shown, shows, successfully, table, the, title, type, updated, was]
TC9	[<<movie.year>>, <<movies>>, <<user>>, <<user>>’s, a, app, button, click, created, deleted, dialog, field, home, in, is, logged, main, message, movie, movies, of, press, profile, screen, search, shown, shows, successfully, table, the, type, updated, was, year]
TC10	[<<user>>, app, button, click, confirm, home, in, is, logged, login, logout, press, screen, shown, successfully, the]

3. Sizes of the test cases				
|TC1| = 20	|TC2| = 21	|TC3| = 36	|TC4| = 41	|TC5| = 41
|TC6| = 40	|TC7| = 37	|TC8| = 45	|TC9| = 34	|TC10| = 16

4. Values for <u>intersections</u> between all pairs			
|TC1 & TC2|= 14	|TC2 & TC6|= 10	|TC4 & TC5|= 32	|TC6 & TC8|= 31
|TC1 & TC3|= 11	|TC2 & TC7|= 9	|TC4 & TC6|= 34	|TC6 & TC9|= 25
|TC1 & TC4|= 11	|TC2 & TC8|= 9	|TC4 & TC7|= 30	|TC6 & TC10|= 13
|TC1 & TC5|= 11	|TC2 & TC9|= 9	|TC4 & TC8|= 30	|TC7 & TC8|= 35
|TC1 & TC6|= 11	|TC2 & TC10|= 7	|TC4 & TC9|= 26	|TC7 & TC9|= 30
|TC1 & TC7|= 11	|TC3 & TC4|= 34	|TC4 & TC10|= 13	|TC7 & TC10|= 13
|TC1 & TC8|= 11	|TC3 & TC5|= 34	|TC5 & TC6|= 35	|TC8 & TC9|= 30
|TC1 & TC9|= 11	|TC3 & TC6|= 31	|TC5 & TC7|= 28	|TC8 & TC10|= 13
|TC1 & TC10|= 10	|TC3 & TC7|= 30	|TC5 & TC8|= 33	|TC9 & TC10|= 13
|TC2 & TC3|= 10	|TC3 & TC8|= 29	|TC5 & TC9|= 25	
|TC2 & TC4|= 10	|TC3 & TC9|= 26	|TC5 & TC10|= 13	
|TC2 & TC5|= 10	|TC3 & TC10|= 13	|TC6 & TC7|= 27	

5. Values for the <u>unions</u> between all pairs of test cases			
|TC1 U TC2 | = 27	|TC2 U TC6 | = 51	|TC4 U TC5 | = 50	|TC6 U TC8 | = 54
|TC1 U TC3 | = 45	|TC2 U TC7 | = 49	|TC4 U TC6 | = 47	|TC6 U TC9 | = 49
|TC1 U TC4 | = 50	|TC2 U TC8 | = 57	|TC4 U TC7 | = 48	|TC6 U TC10 | = 43
|TC1 U TC5 | = 50	|TC2 U TC9 | = 46	|TC4 U TC8 | = 56	|TC7 U TC8 | = 47
|TC1 U TC6 | = 49	|TC2 U TC10 | = 30	|TC4 U TC9 | = 49	|TC7 U TC9 | = 41
|TC1 U TC7 | = 46	|TC3 U TC4 | = 43	|TC4 U TC10 | = 44	|TC7 U TC10 | = 40
|TC1 U TC8 | = 54	|TC3 U TC5 | = 43	|TC5 U TC6 | = 46	|TC8 U TC9 | = 49
|TC1 U TC9 | = 43	|TC3 U TC6 | = 45	|TC5 U TC7 | = 50	|TC8 U TC10 | = 48
|TC1 U TC10 | = 26	|TC3 U TC7 | = 43	|TC5 U TC8 | = 53	|TC9 U TC10 | = 37
|TC2 U TC3 | = 47	|TC3 U TC8 | = 52	|TC5 U TC9 | = 50	
|TC2 U TC4 | = 52	|TC3 U TC9 | = 44	|TC5 U TC10 | = 44	
|TC2 U TC5 | = 52	|TC3 U TC10 | = 39	|TC6 U TC7 | = 50	